# Intro to neural networks

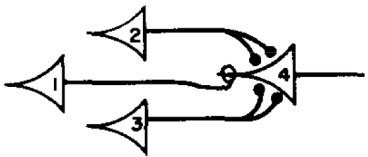
David Puelz
The University of Austin

# Introduction to neural networks

- Neural nets = fancy regression models, the same kind we have been fitting all term
- Loosely inspired by the human brain
- Composed of interconnected "neurons"
- In common NNs, each "neuron" is, literally, a GLM! (Think logistic regression.)
- Applied to a wide range of tasks: images, text, speech, and the chatbots you use
- Typically fit by **stochastic gradient descent**:
  - see a small batch of data
  - make a small correction to the model based on your errors
  - keep going until convergence

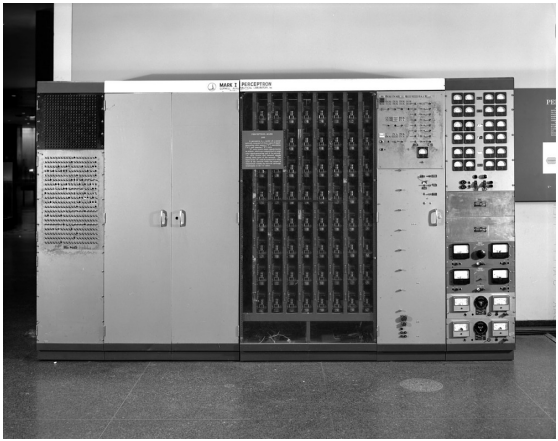
# Brief history of neural networks

- 1943: McCulloch–Pitts neuron model
- 1958: Perceptron by Frank Rosenblatt
- 1986: Backpropagation popularized by Rumelhart, Hinton, and Williams
- 1990s–2000s: Support vector machines and other methods gain popularity
- 2012: Deep learning revolution: AlexNet wins the ImageNet competition
- 2017: Transformers, the architecture behind today's language models



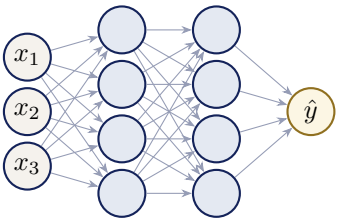
# The original perceptron

- Proposed by Frank Rosenblatt in 1957
- First implementation: the Mark I Perceptron (1958), now at the Smithsonian
- Early attempt at creating artificial intelligence
- Designed to recognize simple patterns: a **linear binary classifier**



# Basic concepts: neurons

- Basic building blocks of neural networks
- Modeled after biological neurons
- Connected by "synapses" (modeled as weights)
- Each neuron is a GLM:
  - it computes a linear function of its inputs
  - it passes the result through an "activation function"
  - it outputs the result to the next neuron(s)



# Typical layer structure

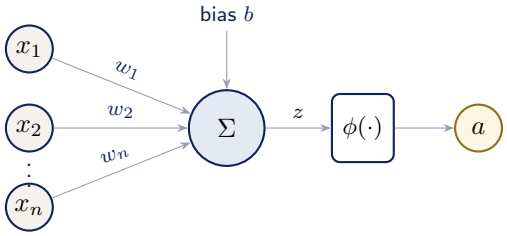
- **Input layer:**
  - Receives input data (features $x$)
  - Number of neurons equals the number of features
  - Passes input data to the first hidden layer
- **Hidden layers:**
  - Perform transformations on input data
  - Can have multiple layers ("deep" learning)
  - Each layer has a varying number of neurons
- **Output layer:**
  - Produces the final output (predictions $\hat{y}$)
  - Number of neurons depends on the task: regression, 1; binary classification, 1; $K$ classes, $K$

# Connecting layers: weights and biases

- **Weights** $(W)$:
  - Connections between neurons in consecutive layers
  - Represented as matrices
  - Example: $W_{jk}^{(\ell)}$ connects neuron $k$ in layer $\ell - 1$ to neuron $j$ in layer $\ell$
- **Biases** $(b)$:
  - Offset values added to neuron inputs: the intercepts
  - Represented as vectors
  - Example: $b_j^{(\ell)}$ is the bias of neuron $j$ in layer $\ell$
- **Activation function** $(\phi)$:
  - Introduces non-linearity into the network
  - Enables learning of complex patterns
  - Without activation functions, networks would be limited to linear transformations

# What happens in a single neuron

- Input values: $x_1, x_2, \ldots, x_n$
- Weights and bias: $w_1, w_2, \ldots, w_n$ and $b$
- Weighted sum: $z = w_1 x_1 + \cdots + w_n x_n + b$
- Activation function: $\phi$
- Output: $a = \phi(z)$



# A single sigmoid neuron is logistic regression

With the sigmoid activation $\phi(z) = 1/(1 + e^{-z})$, the neuron is $P(Y = 1) = \phi(b + wx)$: the logistic regression model from a few weeks ago. On the OkCupid data:

```
glm(male ~ height, data=ok, family=binomial)
nnet(male ~ height, data=ok, size=0, skip=TRUE, entropy=TRUE)   # one neuron
```

| | intercept | height |
|---|---|---|
| `glm` (logistic regression) | -44.449 | 0.6619 |
| `nnet` (one neuron) | -44.449 | 0.6619 |

# Mathematical representation of FFNN architecture

- Initial ("input") layer: $a^{(0)} = x = (x_1, x_2, \ldots, x_n)$
- Hidden layers $\ell = 1, \ldots, L-1$, each a vector of neuron outputs $a^{(\ell)}$
- Final ("output") layer: $a^{(L)} = \hat{y}$
- Weights: $W^{(1)}, W^{(2)}, \ldots, W^{(L)}$; Biases: $b^{(1)}, b^{(2)}, \ldots, b^{(L)}$

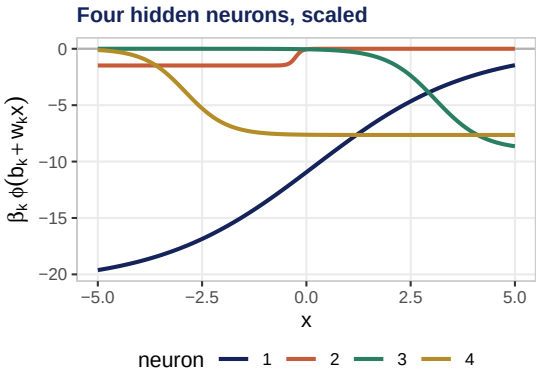
In layer $\ell$:

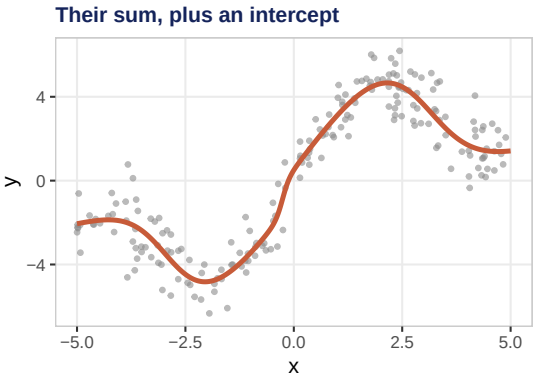
$$z_j^{(\ell)} = \sum_k W_{jk}^{(\ell)} \, a_k^{(\ell-1)} + b_j^{(\ell)}$$

$$a_j^{(\ell)} = \phi\big(z_j^{(\ell)}\big)$$

With one hidden layer of $K$ neurons: $\hat{y} = \beta_0 + \sum_{k=1}^{K} \beta_k \, \phi(b_k + w_k^\top x)$. A regression on $K$ features the network **learns**, where in the nonlinear-regression deck we picked them $(x, x^2, x^3)$.

# Hidden neurons are building blocks



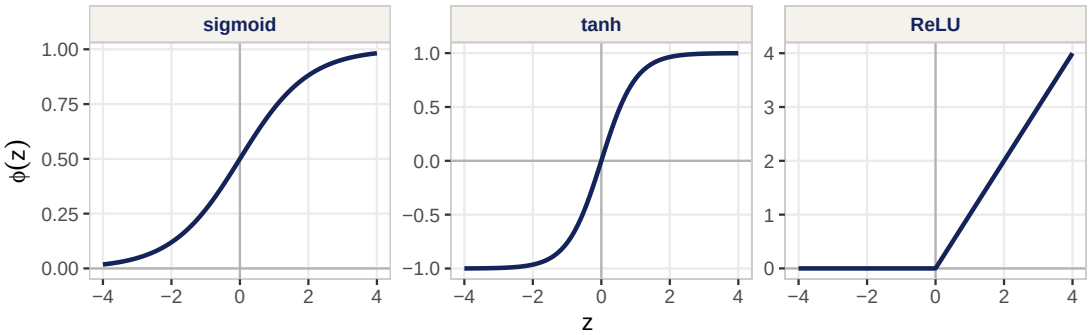




The wave $y = 3 \sin x + x$ from `basic_nn.ipynb`, fit with four sigmoid neurons. Each contributes one S-shaped piece; the network adds them up. With enough neurons, sums like this can approximate any smooth curve.

# Common activation functions

- **Sigmoid:** $\phi(z) = \frac{1}{1+e^{-z}}$
  - Maps input to a value between 0 and 1: the logistic function from logistic regression
  - Historically popular, but can suffer from the vanishing gradient problem
- **ReLU** (Rectified Linear Unit): $\phi(z) = \max(0, z)$
  - Fast to compute, helps mitigate the vanishing gradient problem
  - Most widely used in deep learning
- **Tanh:** $\phi(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
  - Maps input to a value between $-1$ and 1
  - Provides stronger gradients than sigmoid
- **Softmax:** $\phi_k(z) = \frac{e^{z_k}}{\sum_j e^{z_j}}$
  - Maps a vector of scores to a probability distribution over multiple classes
  - Used in multi-class classification: multinomial logistic regression

# Common activation functions




Why we need them: drop $\phi$ and two layers collapse into one linear regression,

$$W^{(2)}\left(W^{(1)}x + b^{(1)}\right) + b^{(2)} = \left(W^{(2)}W^{(1)}\right)x + \left(W^{(2)}b^{(1)} + b^{(2)}\right).$$

# Cost function

- Measures the difference between the predicted output and the actual output
- Objective: minimize the cost function
- Common cost functions:
  - Mean squared error (MSE): $\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$. What `lm` minimizes.
  - Cross-entropy: $-\sum_{i=1}^{n}\left[\, y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\,\right]$. To a statistician, the binomial negative log-likelihood: what `glm(family=binomial)` minimizes.
- Choice of cost function depends on problem type and model architecture

# Making predictions: forward propagation

1. Calculate weighted sum:
$$z_j^{(\ell)} = \sum_k W_{jk}^{(\ell)} \, a_k^{(\ell-1)} + b_j^{(\ell)}$$
2. Apply activation function:
$$a_j^{(\ell)} = \phi\big(z_j^{(\ell)}\big)$$
3. Propagate recursively from the input layer to the output layer

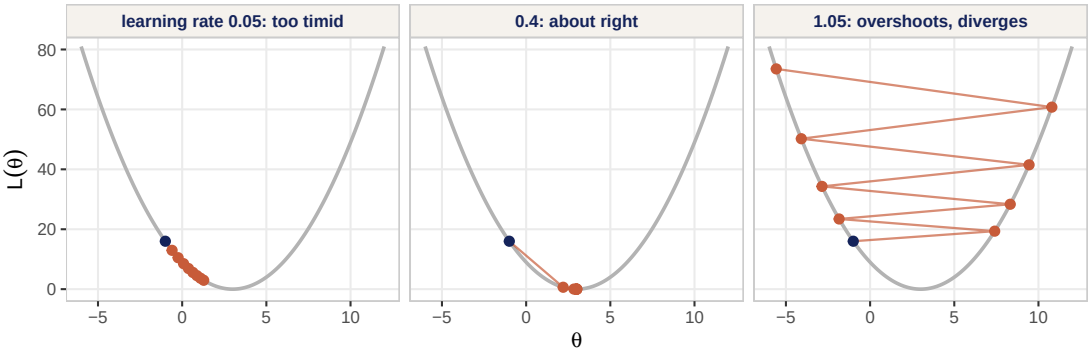
# Gradient descent optimization

- Algorithm to minimize the cost function
- Adjusts weights and biases iteratively: "intelligent trial and error"
- Updates are based on the gradient of the cost function with respect to the parameters:

$$\theta \leftarrow \theta - \eta \nabla L(\theta)$$

- Learning rate $\eta$: step size for each update
- Variants: stochastic gradient descent (SGD), minibatch gradient descent, and adaptive versions like Adam

# Gradient descent optimization




Eight steps on $L(\theta) = (\theta - 3)^2$, each run starting at the navy dot. The learning rate is a tuning parameter, like $\lambda$ in penalized regression.

# Backpropagation algorithm

- Calculates gradients for weights and biases
- Efficient technique for computing partial derivatives
- Developed in the 1960s and 70s by multiple researchers
- Key contributors: Henry J. Kelley, Arthur E. Bryson, Stuart Dreyfus, and Seppo Linnainmaa
- Popularized for neural networks by Rumelhart, Hinton, and Williams in 1986

# Backpropagation algorithm

- Chain rule of calculus forms the basis of backpropagation
- Given a composite function $y = f(g(h(x)))$, the chain rule states:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx} \quad \text{where} \quad u = g(h(x)),\ v = h(x)$$

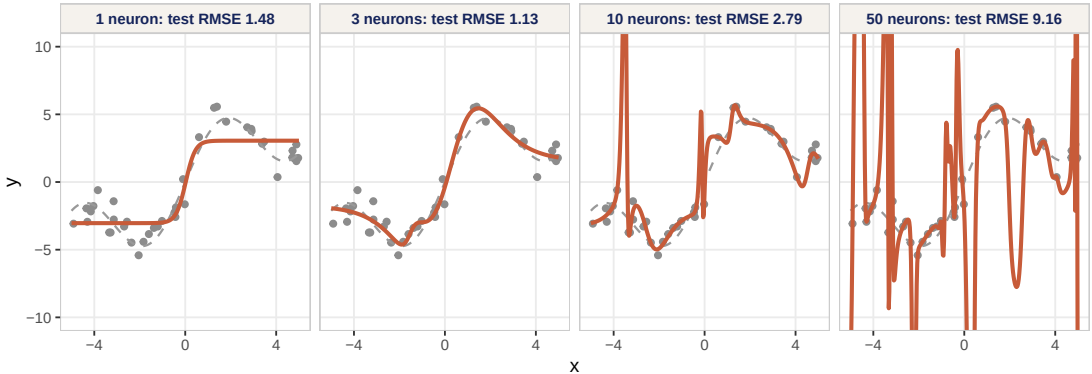
- In backpropagation, the chain rule is applied to compute gradients for weights and biases:
  1. Perform forward propagation to compute the predicted output
  2. Compute the error between predicted and actual output
  3. Calculate gradients for output layer weights and biases
  4. Propagate gradients backward through the network, layer by layer
  5. Update weights and biases using the computed gradients

# Basic neural network training loop

**Inputs:** number of epochs, minibatch size, learning rate

```
1. Initialize network parameters (weights and biases)
2. for epoch in range(number of epochs):
3.     Shuffle the dataset
4.     for minibatch in dataset.split(minibatch_size):
5.         Initialize gradients to zero
6.         for sample in minibatch:
7.             Perform forward propagation to compute output
8.             Compute the error between predicted and actual output
9.             Perform backward propagation to compute gradients
10.            Accumulate gradients for the minibatch
11.        Update network parameters using accumulated gradients:
           W = W - learning_rate * (1/minibatch_size) * sum(gradients)
```

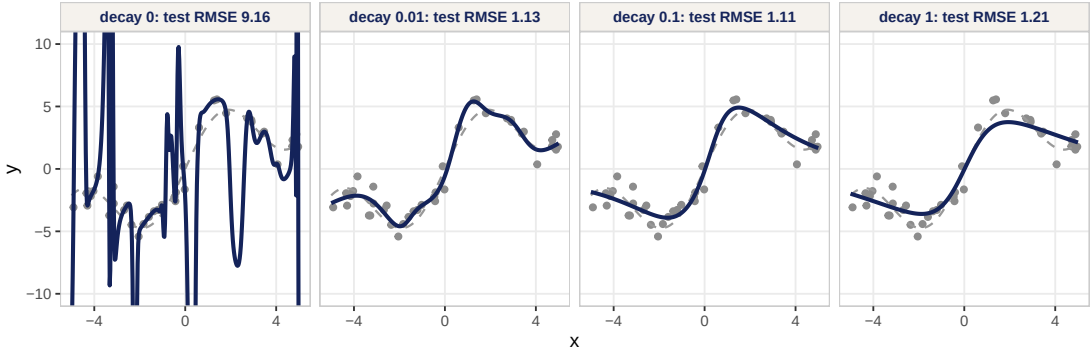
# Overfitting: more neurons, more flexibility




Forty noisy points from the wave (dashed). Fifty hidden neurons fit the training data almost perfectly (RMSE 0.002) and new data terribly. The bias-variance tradeoff from week 1, again.

# Regularization: weight decay is ridge regression

Add the ridge penalty from penalized regression to the cost function: $L(\theta) + \lambda \sum_j \theta_j^2$.




Same 50 neurons, now with `nnet(..., decay=`$\lambda$`)`. Shrinking the weights tames the wiggles.
Other tools: early stopping, dropout, more data.

# Common software environments

- Several popular software environments for training neural networks:
  - **PyTorch**, from Meta (now run by the PyTorch Foundation)
  - **JAX**, from Google
  - **TensorFlow**, from Google
  - **Keras 3**, a high-level interface that runs on PyTorch, JAX, or TensorFlow
  - In R: `nnet` for small networks like today's, and the `torch` package
- All have their strengths and are widely used in the ML community
- I recommend and use **PyTorch** for its:
  - Ease of use and intuitive syntax
  - Dynamic computation graph
  - Strong community support and extensive documentation
- **It just feels a lot more like writing plain Python than other frameworks.**

# Examples

Let's see two quick examples:

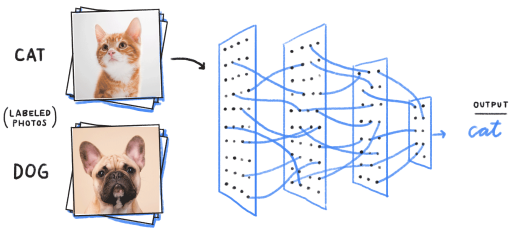
- `basic_nn.ipynb`: the wave from today's slides
- `classification_nn.ipynb`: an easy and a hard two-class problem

# Convolutional neural networks for images

- Convolutional neural networks (CNNs) are a type of deep learning model specifically designed to work with images.
- They can be used for tasks such as:
  - image classification
  - object detection
  - semantic segmentation
- CNNs were really the start of the modern deep learning revolution (AlexNet, 2012)!

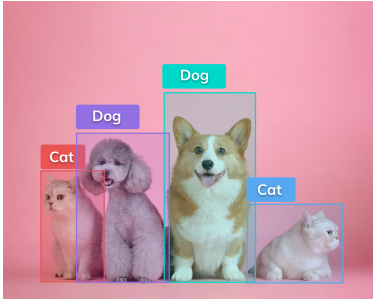
# Image classification

- Assign an input image to one of several predefined categories
- Commonly used for tasks such as digit recognition and scene classification
- Requires a labeled dataset for training



# Object detection

- Identify and locate objects within an image
- Provide bounding boxes around each detected object
- Combine image classification with localization techniques
- Useful for applications like autonomous vehicles and medical imaging



# Semantic segmentation

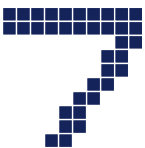
- Assign a class label to each pixel in the input image
- Understand the context of the image at the pixel level
- Useful for tasks like autonomous driving, medical image analysis, and robotics



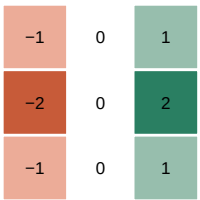
# Convolutional layers

- Key component of CNNs
- Apply filters to the input image: multiply each small patch by the filter and add
- Detect features such as edges, shapes, and textures. The filter's numbers are weights the network learns.
- Multiple filters allow the network to learn different features

input image



3 x 3 edge filter

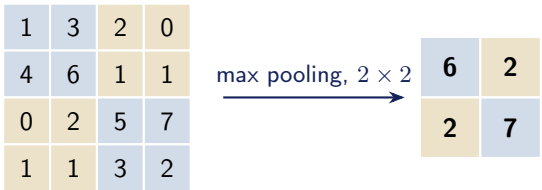



feature map: vertical edges



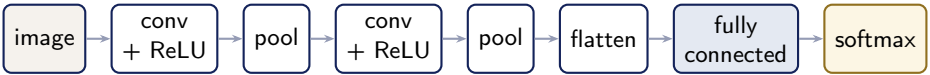
# Pooling layers

- Reduce the spatial dimensions of the feature maps
- Make the network more resistant to small changes in input
- Common types: max pooling, average pooling



# Fully connected layers

- Flatten the feature maps into a single vector
- Perform classification or regression tasks
- Utilize activation functions (e.g., ReLU, softmax) to introduce non-linearity
- You've seen these before! They are the networks from the first half of today.



# Examples of CNN architectures

- LeNet-5 (1998): early CNN architecture for digit recognition
- AlexNet (2012): breakthrough architecture, won the ImageNet competition
- VGGNet (2014): improved accuracy with deeper networks
- ResNet (2015): enabled training of very deep networks using residual connections

| network | year | layers | ImageNet top-5 error |
|---|---|---|---|
| AlexNet | 2012 | 8 | 15.3% |
| VGGNet | 2014 | 16–19 | 7.3% |
| ResNet | 2015 | 152 | 3.6% |

# Examples

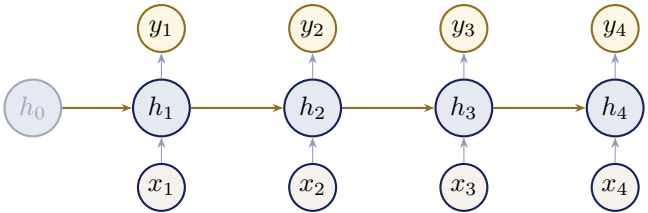
Let's see a quick example in `MNIST.ipynb`, where we'll train a conv-net to recognize hand-written digits (0–9).

# Recurrent neural networks for sequences

- Recurrent neural networks (RNNs) are designed for sequential data, such as time series, natural language text, and audio signals.
- They can handle inputs and outputs of varying lengths, making them suitable for tasks like language translation, sentiment analysis, and speech recognition.
- They are no longer cutting edge! Since 2017 the best sequence models are based on **transformers** (the "T" in GPT), which power today's chatbots. Still important to know RNNs: they are the time series regressions of deep learning.

# Basic RNN structure

- Process input sequences one element at a time
- Maintain a hidden state vector to store information from previous time steps
- Update the hidden state as new input elements are processed



# RNN math

An RNN updates its hidden state vector recursively as follows:

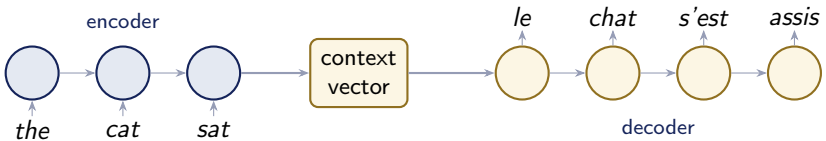
- Input vector: $x_t$; Hidden state vector: $h_t$
- Weight matrices: $W_{xh}$, $W_{hh}$, $W_{hy}$
- Bias vectors: $b_h$, $b_y$
- Update/predict equations:

$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$
$$y_t = \text{predict}(h_t)$$

- When you learn an RNN, you're learning those weight matrices, shared across time steps
- Compare AR(1) from time series regression: $y_t = \beta_0 + \beta_1 y_{t-1} + \epsilon_t$
- You can also stack RNNs to make deeper models, e.g. layer 2's input is layer 1's output

# Sequence-to-sequence prediction

- Encoder processes the input sequence and generates a "context vector", representing everything seen up to that point.
- Decoder uses the context vector to produce the output sequence, one item at a time. "Autocomplete on steroids."
- RNNs can be used for tasks like machine translation, speech recognition, and image captioning.



# Challenges with RNNs

- Vanishing gradients: difficulty in learning long-range dependencies. Backpropagating through 50 steps multiplies by roughly the same factor 50 times: $0.9^{50} \approx 0.005$.
- Exploding gradients: numerical instability during training. $1.1^{50} \approx 117$.
- One solution: Long Short-Term Memory (LSTM) networks
  - Solve the vanishing gradient problem using memory cells and gating mechanisms
  - Preserve information across long sequences
  - More complex and computationally expensive than basic RNNs
- Another solution: Gated Recurrent Unit (GRU)
  - Simplified version of the LSTM
  - Uses update and reset gates
  - Requires fewer parameters and is faster to train

# GRUs

- Gated recurrent units (GRUs) are a type of recurrent neural network (RNN)
- They are designed to help with the vanishing gradient problem that can occur in traditional RNNs
- Like RNNs, GRUs process sequences of input data and use a hidden state to retain information from previous inputs
- However, GRUs have an additional gating mechanism that helps them selectively forget or remember information from the past, making them more effective than traditional RNNs
- Each gate is itself a small layer of sigmoid neurons: logistic regressions deciding how much to keep

# Math of a GRU unit

- GRUs have two gates, an update gate $z$ and a reset gate $r$, which control the flow of information through the unit.
- The output $h_t$ is a combination of the previous hidden state $h_{t-1}$ and the current input $x_t$.

$$\begin{aligned}
\text{update gate:} \quad & z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \\
\text{reset gate:} \quad & r_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \\
\text{candidate hidden state:} \quad & \tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h) \\
\text{final hidden state:} \quad & h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t
\end{aligned}$$

- $W_z$, $W_r$, $W_h$ are weight matrices, $b_z$, $b_r$, $b_h$ are bias vectors, $\sigma$ is the sigmoid, and $\odot$ is element-wise multiplication.

# Example: sequence reversal

We'll look at a fun example in `seq_to_seq_toy.ipynb`, where we'll train a seq-to-seq model that learns to reverse a given input word.

Given a sequence of characters from a word with its characters in reverse order, the model should generate the correct forward sequence of characters representing the original word.

For example, if the input is "yekrut", the output should be "turkey":

```
input_seq              => output_seq
y - e - k - r - u - t  => t - u - r - k - e - y
```

Of course this is silly!

But the problem our seq2seq model will solve, i.e., reversing words, shares similarities with more complex tasks like machine translation and natural language generation, as tackled by models like ChatGPT and Claude.

While the word reversal problem is *much* simpler, it serves as a stepping stone to understanding the underlying concepts and techniques that apply to those more advanced tasks.

# Key concepts illustrated

- **Encoder-decoder architecture**: the encoder processes the input sequence and generates a context vector (hidden state) that represents the input. The decoder then takes this context vector and generates the output sequence.
- **Recurrent neural networks**: RNNs are suitable for sequence-to-sequence learning tasks because they can handle sequences of varying lengths and capture temporal dependencies in the data.
- **One-hot encoding**: the input characters are represented as binary vectors, the same idea as the dummy variables from categorical predictors.
- **Custom PyTorch dataset and data loading**: the model uses a custom PyTorch dataset (`ReverseWordDataset`) to handle loading, encoding, and decoding of training data.